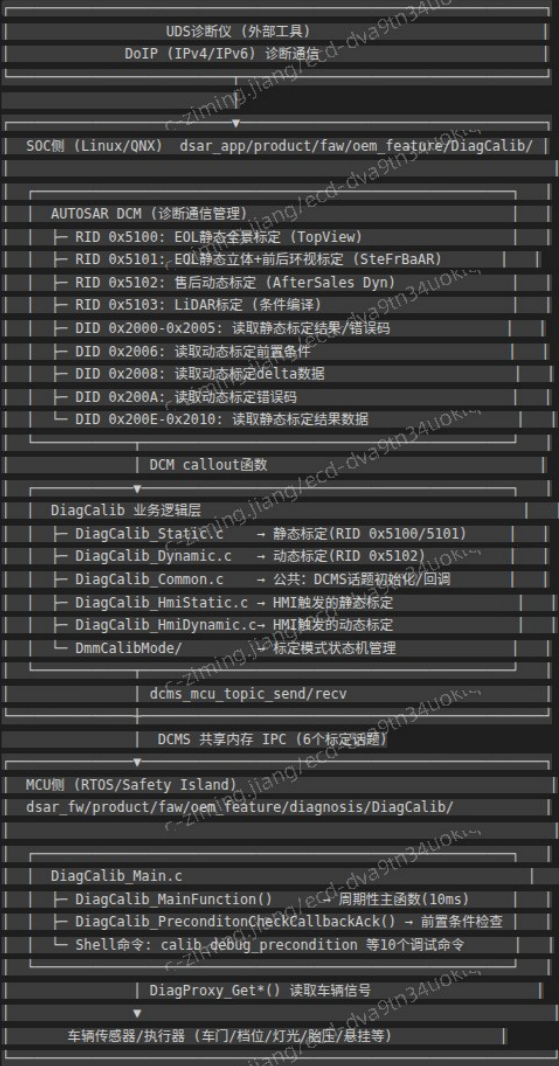


标定功能架构设计与调用关系

一、整体架构

标定系统是一个跨SOC/MCU的诊断标定框架，通过UDS诊断协议（DoIP）对外暴露标定服务，内部分为三大部分：



二、核心DCMS话题（6个话题，跨SOC/MCU通信）

所有SOC → MCU通信通过DCMS共享内存话题实现：

话题名称	方向	用途
DCMS_TOPIC_CALIB_PRECONDITION_CHECK	SOC → MCU	请求前置条件检查(静态/动态)
DCMS_SUB_EOLCALIB_RESULT_RTOS	MCU → SOC	标定结果状态(1s周期推送)
DCMS_CALIB_DATA_ACK_RTOS	MCU → SOC	标定结果delta数据(按请求)
DCMS_CLIENT_DMM_CALIB_MODE_CMD	SOC → MCU	DMM标定模式切换命令
DCMS_TOPIC_DIAG_AVM_GESTURE	MCU → SOC	HMI手势/触摸坐标
DCMS_TOPIC_AVM_STATE_INFO	MCU → SOC	AVM全景影像状态
DCMS_CLIENT_START_EOLCALIB_RTOS	SOC → MCU	触发感知引擎开始标定
DCMS_CLINET_GETCALIB_PARAM_RTOS	SOC → MCU	请求标定参数数据
DCMS_TOPIC_HMI_CALIB_REQ	SOC → MCU	HMI标定请求/结果通知DMM

话题注册在 DiagCalib_Common.c 的 DiagCalib_CalibrationTopicInit() 中。

三、三层状态机设计

1. DCM诊断层状态机（AUTOSAR标准）

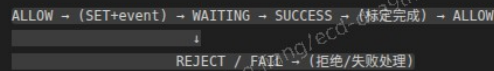
所有RID callout函数遵循AUTOSAR DCM三态机制：

DCM_INITIAL → 发送请求（前置条件检查/标定开始）→ DCM_E_PENDING
DCM_PENDING → 轮询检查结果（每300ms）→ 继续PENDING / 成功返回E_OK / 超时返回E_NOT_OK

- 静态标定超时: 20秒
- 动态标定超时: 20秒(set mode) + 单独的超时处理

2. DMM标定模式状态机

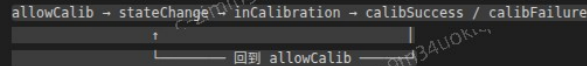
定义在 DmmCalibMode_Type.h:



DMM负责协调标定模式切换（如进入/退出标定模式），MCU侧根据车辆状态决定是否允许。

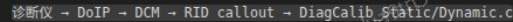
3. HMI标定状态机

定义在 DiagCalib_HmiCalibType.h:



四、两种标定触发方式

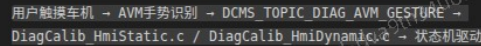
方式A: UDS诊断仪触发（EOL产线）



以RID 0x5100（静态全景标定）为例：

- 1. DCM_INITIAL : 发送前置条件检查话题到MCU → 设置DMM模式为START_STATIC_CALIB → 返回PENDING
- 2. DCM_PENDING : 等待1秒后每300ms轮询 DMM模式状态 + 前置条件
- 3. 两者都OK → DiagCalib_CalibStartTopicSend() 发送标定开始话题到感知引擎
- 4. 感知引擎执行标定 → 结果通过 DCMS_SUB_EOLCALIB_RESULT_RTOS 推送回SOC
- 5. 诊断仪通过DID 0x2000-0x2005读取结果

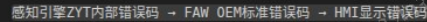
方式B: HMI车机触摸触发（售后）



关键入口条件（以E001/E202为例）：

- 进入标定页面：在坐标区域（822~1022，0~200）内连续触摸4次且在3秒内
- 开始标定：在（398~498，674~1126）— 静态 / （524~624，474~926）— 动态
- 停止标定：在（848~948，474~926）
- 必须AVM界面已启动（AvmState == 1）
- 互斥锁 HmiCalibReqMutex 保证静态/动态不会同时触发

五、三层错误码翻译链



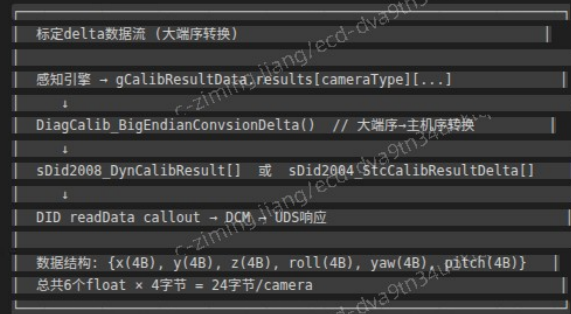
第一层（ZYT→FAW），在 DynCalib_ResultStatusCheck() DiagCalib_Dynamic.c:26 / StaticCalib_ResultStatusCheck() 中：

ZYT错误码	FAW OEM错误码
ZYT_Normal_ERRORCODE	FAW_DYN_Normal_ERRORCODE
ZYT_sInProcess_ERROECODE	FAW_DYN_SInProcess_ERROECODE
ZYT_NoImage_ERRORCODE	FAW_DYN_NoImage_ERRORCODE
... (共12种映射)	

第二层（FAW→HMI），在 HmiDynCalib_CalibResultStatusCheck() DiagCalib_HmiDynamic.c:67 / HmiStaticCalib_HmiIndexSendCalibResultToDmmTx() 中：

FAW OEM	HMI
FAW_DYN_Normal_ERRORCODE	Hmi_FAW_CalibSuccess (0x3)
FAW_DYN_NoImage_ERRORCODE	Hmi_NoImage_ERRORCODE (0x4)
... (共14种映射)	

六、关键数据流



七、MCU侧：前置条件检查

DiagCalib_PreconditionCheckCallbackAck() DiagCalib_Main.c 通过 DiagProxy_Get*() 系列函数读取车辆信号：

静态标定前置条件：所有车门关闭、P档、近光灯/位置灯关闭、胎压正常、无主动悬挂故障

动态标定前置条件：所有车门关闭、非静止、车速>0、空气悬挂正常、胎压正常

调试标志 gPreconditionCalibDebug 可绕过所有前置条件检查。

八、文件清单与职责

文件	职责
DiagCalib_CommonType.h	所有标定数据结构、枚举、错误码定义
DiagCalib_Common.c	DCM/ECU通信、标定开始话题推送、回调逻辑

DiagCalib_Common.c	DCMS话题注册、标定开始话题发送、回调路由
DiagCalib_Static.c	RID 0x5100/5101 callout、静态结果处理、delta解析
DiagCalib_Dynamic.c	RID 0x5102 callout、动态结果处理、delta请求
DiagCalib_HmiStatic.c	HMI静态标定手势识别、状态机
DiagCalib_HmiDynamic.c	HMI动态标定手势识别、状态机、错误码 → HMI映射
DiagCalib_HmiCalibType.h	HMI坐标常量（4车型×2标定类型）、触摸超时配置
DmmCalibMode/DmmCalibMode.c	DMM模式SET/QUERY命令及回调
DiagCalib_Cfg.h	传感器配置映射表(错误码索引、delta数据索引)
MCU侧	
DiagCalib_Main.c	MCU主函数、前置条件检查、Shell调试命令
DiagCalib_Main.h	MCU侧数据结构(前置条件定义等)

九、关键设计模式

1. DCM Callout模式：所有RID/DID处理函数以AUTOSAR标准 FUNC(Std_ReturnType, DCM_CALLOUT_CODE) 声明，由DCM配置在诊断描述文件中注册。函数名由DCM配置静态绑定。
2. 异步请求-回调模式：SOC发送DCMS话题到MCU后返回 DCM_E_PENDING，MCU处理完成后通过回调话题通知SOC，SOC在下次DCM轮询时读取结果。
3. 车型差异化：通过 Diag_GetCarModelValue() 区分车型（30=E001, 31=E009, 32=E202, 33=E007），HMI坐标常量按车型分组定义。
4. 互斥锁：HmiCalibReqMutex 保证静态标定、动态标定、停止标定三者不会并发执行。
5. 超时保护：每一层都有独立超时——DCM callout有 DiagCalib_SetModeTimeOutFlag()/GetDidTimeOutFlag()，HMI状态机有各阶段超时（静态150s、动态1000s），避免死等。

Update Todos

- ✓ Read DiagCalib header files (types, configs, interfaces) on SOC-side
- ✓ Read DiagCalib implementation files on SOC-side
- ✓ Read DmmCalibMode and MCU-side files
- ✓ Trace call relationships and data flow across SOC-MCU boundary
- ✓ Summarize architecture design and key design patterns

Thought for 1s

以上就是标定功能的完整架构设计与调用关系。核心要点：

- 跨芯片通信：SOC侧处理UDS诊断协议和HMI交互，MCU侧处理车辆信号前置条件检查，通过9个DCMS话题在共享内存上通信
- 三层状态机：DCM诊断层(AUTOSAR标准三态) → DMM标定模式管理层 → HMI用户交互层
- 双触发方式：产线用UDS诊断仪，售后用车机触摸，坐标参数按车型（E001/E009/E202/E007）区分
- 三层错误码翻译：感知引擎(ZYT内部码) → FAW OEM标准码 → HMI显示码

soc calib侧和mcu前置条件侧是如何通信的，架构如何设计的

Thinking...

Interrupted

soc calib侧和mcu前置条件侧是如何通信的，架构如何设计的，调用链是怎样的

Thought for 2s

Let me trace the exact call chain through the code.

Read DiagCalib_Common.c

Read DiagCalib_Static.c

Thought for 1s

Now let me read the MCU side to complete the communication chain.

Read DiagCalib_Main.c

Read DiagCalib_Common.c

Thought for 31s

现在我已看到完整的通信代码。以下是 SOC 标定侧与 MCU 前置条件侧之间的通信架构和调用链。

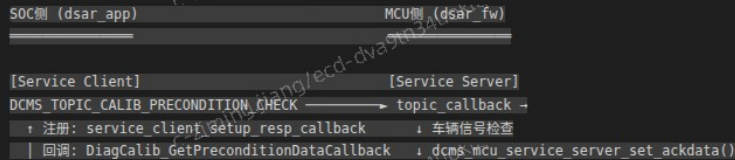
SOC ↔ MCU 标定通信架构

一、通信基础：DCMS 两种模式

DCMS 提供两种 IPC 模式：

模式	API	方向	用途
Topic (话题)	dcms_mcu_topic_send / dcms_mcu_topic_setup_callback	单向推送/订阅	标定结果状态(1s周期)、HMI手势、AVM状态
Service (服务)	dcms_mcu_topic_send → dcms_mcu_service_server_set_ackdata → dcms_mcu_service_client_setup_resp_callback	请求-应答	前置条件检查、DMM模式切换

二、SOC/MCU 两侧注册的话题



```
|
|
|  ← 应答
|
[Service Client] [Service Client]
DCMS_CLIENT_DMM_CALIB_MODE_CMD → topic_callback
↑ 注册: service_client_setup_resp_callback ↓ 模式切换
| 回调: DiagCalib_DmmModeStatusCallback | dcms_mcu_service_server_set_ackdata()
|
|  ← 应答
|
```

```
[Topic Subscriber] [Topic Publisher] (感知引擎推送)
DCMS_SUB_EOLCALIB_RESULT_RTOS ← 标定结果状态(1s周期)
| 回调: DiagCalib_CalibResultStatusCallback | MCU侧也有同名校验回调
```

```
[Topic Subscriber]
DCMS_CALIB_DATA_ACK_RTOS ← 标定delta数据(按需)
| 回调: DiagCalib_CalibResultDataCallback
```

```
[Topic Subscriber] [Topic Publisher] (座舱域控/AVM推送)
DCMS_TOPIC_DIAG_AVM_GESTURE ← HMI触摸手势坐标
| 回调: DiagCalib_GetHmiCalibRequestCallback
```

```
[Topic Subscriber]
DCMS_TOPIC_AVM_STATE_INFO ← AVM全景影像状态
| 回调: DiagCalib_GetAvmInfoCallback
```

```
[Topic Publisher] [Topic Subscriber]
DCMS_CLIENT_START_EOLCALIB_RTOS ← 感知引擎接收标定开始/停止指令
| DiagCalib_CalibStartTopicSend()
```

```
[Topic Publisher] [Topic Subscriber]
DCMS_CLINET_GETCALIB_PARAM_RTOS ← 感知引擎返回标定参数
| StaticCalib_ResultDataRequest()
| DynCalib_ResultDataRequest()
```

三、完整调用链：以 RID 0x5100 静态全景标定为例

时间线 →

① DCM_INITIAL (SOC)

```
外部诊断仪 → DoIP → AUTOSAR DCM
└─ eol_topViewCalibEol_requestStart_routine() [DiagCalib_Static.c:932]
|
| 步骤1: 发送前置条件检查请求到MCU
|   dcms_mcu_topic_send(DCMS_TOPIC_CALIB_PRECONDITION_CHECK, {reqPreconditionType=REQ_STATIC})
|
| 步骤2: 设置DMM标定模式
|   DiagCalib_DmmModeCmd(DSAR_DMM_CALIB_MODE_SET, DMM_EVENT_START_STATIC_CALIB)
|   └─ dcms_mcu_topic_send(DCMS_CLIENT_DMM_CALIB_MODE_CMD, {SET, START_STATIC})
|
└─ 返回 DCM_E_PENDING
```

② MCU侧处理 (异步)

```
[DiagCalib_Main.c:69]
DiagCalib_PreconditionCheckCallbackAck() 被DCM回调触发:
|
|─ DiagProxy_GetFLDoorStatus() // 左前门
|─ DiagProxy_GetFRDoorStatus() // 右前门
|─ DiagProxy_GetRLDoorStatus() // 左后门
|─ DiagProxy_GetRRDoorStatus() // 右后门
|─ DiagProxy_GetTailgateDoorSt() // 尾门
|─ DiagProxy_GetFrontHoodStatus() // 前舱盖
|─ DiagProxy_GetLeverInfo() // 档位(需为M档)
|─ DiagProxy_GetHighBeamSwtSt() // 远光
|─ DiagProxy_GetLowBeamSt() // 近光
|─ DiagProxy_GetTurnLitLeftSt() // 左转向灯
|─ DiagProxy_GetTurnLitRightSt() // 右转向灯
|─ DiagProxy_GetPositionLitswtSt() // 廓灯
|─ DiagProxy_GetTPMS_FLTPMSError() // 左前胎压
|─ DiagProxy_GetTPMS_FRTPMSError() // 右前胎压
|─ DiagProxy_GetTPMS_RLTPMSError() // 左后胎压
|─ DiagProxy_GetTPMS_RRTPMSError() // 右后胎压
|─ DiagProxy_GetActiveSuspensionInfo() // 主动悬架
|
|─ 逐位检查 → respResult.respPreconditionStatus
|   Static_FourDoorTwoCovers_PosBit - 四门两盖任一未关
|   Static_GearNot_N_PosBit - 档位不在N
|   Static_TirePressAbnormal_PosBit - 胎压异常
|   Static_ActiveSuspensionAbnormal_PosBit - 悬架异常
|
|─ debug旁路: if (gPreconditionCalibDebug) → result = 0
|
└─ dcms_mcu_service_server_set_ackdata(DCMS_TOPIC_CALIB_PRECONDITION_CHECK, &respResult)
   ↑ 通过Service模式将结果回调给SOC侧
```

③ DCM_PENDING (SOC, 每10ms触发一次, 1秒后开始检查)

```
eol_topViewCalibEol_requestStart_routine() 再次被DCM调用:
|
|─ 等待 sStaticCalibSetModeTime > 1000ms 且 每300ms轮询
```



```
└─ 检查前置条件:
    DiagCalib_DmmModeCmd(QUERY, 0) → 查询DMM模式状态
    gPreconditionCalibStatus.getPreconditionStatus == 0? → 通过回调已更新
    DiagCalib_DmmModeStatusGet() == DSAR_DMM_CALIB_SUCCESS?
└─ 都OK → 步骤4
└─ 前置条件!=0 → 返回 DCM_E_CONDITIONSNOTCORRECT (NRC 0x22)
└─ DMM还在WAITING/ALLOW → 继续 DCM_E_PENDING
└─ 其他 → 返回 DCM_E_GENERALREJECT (NRC 0x10)

└─ 超时20秒 → DiagCalib_SetModeTimeOutFlag() → 返回失败

④ 触发感知引擎标定
    DiagCalib_CalibStartTopicsSend(kCalibIdxStcTopView) [DiagCalib_Common.c:78]
    └─ dcms_mcu_topic_send(DCMS_CLIENT_START_EOLCALIB_RTOS,
        {sensortime, calibtype=kCalibIdxStcTopView, detail_type=StaCalib_NoParam})
    └─

⑤ 感知引擎执行标定 → 结果通过 DCMS_SUB_EOLCALIB_RESULT_RTOS (1s周期) 推回

⑥ SOC侧回调处理
    DiagCalib_CalibResultStatusCallback() [DiagCalib_Common.c:125]
    └─ 根据 gCurCalibCamera 路由:
        └─ kCalibIdxStcTopView → StaticCalib_ResultStatusCheck() [DiagCalib_Static.c:154]
            └─ ZYT错误码 → FAW_OEM错误码 → DiagCalib_ParseSensorMsg() → RID结果数组
        └─ kCalibIdxDynaHw → DynCalib_ResultStatusCheck()

⑦ 诊断仪读取结果
    DID 0x2000 → eol_topview_result_readData() → 返回 delta位姿数据
    DID 0x2002 → eol_topview_errorCode_readData() → 返回 各相机错误码
    RID 0x5100 requestResults → eol_topviewCalibEol_requestResults_routine() → 返回总结果
```

四、SOC侧数据接收流程

DCMS话题	SOC侧回调函数	数据落点
DCMS_TOPIC_CALIB_PRECONDITION_CHECK (service client resp callback)	DiagCalib_GetPreconditionDataCallback [Common.c:195]	gPreconditionCalibStatus .getPreconditionStatus .getFlag = true
DCMS_SUB_EOLCALIB_RESULT_RTOS (topic callback, 1s周期)	DiagCalib_CalibResultStatusCallback [Common.c:125]	gCalibResultStatus .results[cameraType][...] └─ StaticCalib_ResultStatusCheck() └─ DynCalib_ResultStatusCheck()
DCMS_CALIB_DATA_ACK_RTOS (topic callback, 按需)	DiagCalib_CalibResultDataCallback [Common.c:158]	gCalibResultData .results[cameraType][...] └─ StaticCalib_ResultDataCheck() gGetCalibDataDeltaFlag = true └─ DynCalib_ResultDataCheck() gGetDynCalibDataDeltaFlag = true
DCMS_CLIENT_DMM_CALIB_MODE_CMD (service client resp callback)	DiagCalib_DmmModeStatusCallback [DmmCalibMode.c:71]	gDmmCalibModeStatus (在SOC侧DmmCalibMode.c中)
DCMS_TOPIC_DIAG_AVM_GESTURE (topic callback)	DiagCalib_GetHmiCalibRequestCallBack [Common.c:232]	gViAVMGestureStrtRx .touchPointCoordinate_X/Y sGetHmiCalibRequestSuccessFlag
DCMS_TOPIC_AVM_STATE_INFO (topic callback)	DiagCalib_GetAvmInfoCallBack [Common.c:248]	AvmStatusInfo.AvmState sGetAvmInfoSuccessFlag

五、关键设计要点

- 异步回调模式: SOC发送DCMS消息后立即返回 DCM_E_PENDING, 不阻塞DCM任务。MCU处理完后通过回调更新全局变量, SOC在后续DCM轮询(每10ms一次)时读取结果。
- SOC/MCU两侧都有相同的全局变量: gCalibResultStatus、gDmmCalibModeStatus、gCurCalibCamera 在SOC和MCU两侧独立维护, 通过DCMS话题同步。SOC侧的来自MCU感知引擎推送, MCU侧的来自感知引擎推送。
- 超时保护三层:
 - DCM callout 超时: DiagCalib_SetModeTimeOutFlag() — 20秒
 - DID readData 超时: DiagCalib_GetDidTimeOutFlag() — 15秒
 - HMI 状态机超时: 静态150s / 动态1000s
- 前置条件按标定类型区分: 同一个 DCMS_TOPIC_CALIB_PRECONDITION_CHECK 话题, 通过 reqPreconditionType 字段区分静态 (REQ_STATIC_PROCONDITION_STATUS) 和动态 (REQ_DYNAMIC_PROCONDITION_STATUS), MCU侧根据类型检查不同的车辆信号组合。